

# Tugas 4

II4021 Kriptografi

## Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi

Disusun oleh

Kelompok 11



Audy Alicia Renatha  
Tirayoh  
(18223097)

Samuel Chris Michael  
Bagasta Simanjuntak  
(18223011)

Carlen Asadel  
Axelle  
(18223017)

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI  
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA  
INSTITUT TEKNOLOGI BANDUNG  
2026**

## Pernyataan Tidak Melakukan Kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.

Samuel Chris Michael  
Bagasta Simanjuntak

Carlen Asadel Axelle

Audy Alicia Renatha Tirayoh

## Daftar Isi

|                                                          |          |
|----------------------------------------------------------|----------|
| Pernyataan Tidak Melakukan Kecurangan.....               | 2        |
| Daftar Isi.....                                          | 2        |
| Daftar Tabel.....                                        | 4        |
| Daftar Gambar.....                                       | 4        |
| <b>I. Pendahuluan.....</b>                               | <b>1</b> |
| <b>II. Dasar Teori.....</b>                              | <b>1</b> |
| II.1 Shamir Secret Sharing.....                          | 1        |
| II.2 Interpolasi Lagrange.....                           | 1        |
| II.3 Enkripsi Simetris AES-128-GCM.....                  | 2        |
| II.4 Key Derivation Function.....                        | 2        |
| II.5 Pembangkit Bilangan Acak Kriptografis (CSPRNG)..... | 2        |
| II.6 Visual Cryptography.....                            | 3        |
| II.7 QR Code.....                                        | 3        |
| <b>III. Perancangan Sistem.....</b>                      | <b>3</b> |
| III.1 Gambaran Umum Sistem.....                          | 3        |
| III.2 Arsitektur Sistem.....                             | 4        |
| III.2.1 Peran Client dan Server.....                     | 4        |
| III.2.2 Alur Akses Mode Normal dan Mode Backup.....      | 5        |
| III.3 Rancangan Database.....                            | 6        |
| III.3.1 Struktur Vault dan Entri Password.....           | 6        |
| III.3.2 Penyimpanan Lokal Klien.....                     | 6        |
| III.3.3 Database Server.....                             | 7        |
| III.4 Rancangan Distribusi Kunci.....                    | 8        |
| III.5 Rancangan Enkripsi.....                            | 9        |
| III.6 Antarmuka CLI dan Integrasi.....                   | 9        |
| III.7 Password Generator.....                            | 10       |
| III.8 QR Code untuk Recovery Share.....                  | 10       |
| III.9 Kriptografi Visual untuk Recovery Share.....       | 10       |

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>IV. Pengujian Program dan Analisis Hasil.....</b>                                  | <b>11</b> |
| IV.1 Skenario Pengujian.....                                                          | 11        |
| IV.2 Uji Pembuatan Vault.....                                                         | 11        |
| IV.2.1 Buat Vault Baru dengan Master Password Valid.....                              | 11        |
| IV.2.2 Pembagian Master Key menjadi Tiga Share.....                                   | 12        |
| IV.2.3 Recovery Share Dalam Format Teks.....                                          | 13        |
| IV.2.4 Server Menyimpan Metadata.....                                                 | 13        |
| IV.3 Uji Penyimpanan dan Perlindungan Local Share.....                                | 13        |
| IV.3.1 Local Share Tidak Plain.....                                                   | 13        |
| IV.3.2 Master Password Benar.....                                                     | 14        |
| IV.3.3 Master Password Salah.....                                                     | 14        |
| IV.4 Uji Akses Normal.....                                                            | 15        |
| IV.4.1 Buka Vault dengan Master Password Benar (Kombinasi Local + Server Share).....  | 15        |
| IV.4.2 Rekonstruksi Master Key dari Dua Share Valid.....                              | 15        |
| IV.4.3 Vault Didekripsi dan Isi Ditampilkan.....                                      | 15        |
| IV.4.4 Master Password Salah atau Share Tidak Valid.....                              | 16        |
| IV.5 Uji Penambahan Data Password.....                                                | 17        |
| IV.5.1 Menambahkan Data Password dengan Input Manual.....                             | 17        |
| IV.5.2 Menambahkan Data Password dengan Pembangkitan Otomatis menggunakan CSPRNG..... | 17        |
| IV.5.3 Enkripsi Ulang dengan Nonce Baru.....                                          | 18        |
| IV.5.4 Backup Lokal Diperbarui.....                                                   | 19        |
| IV.6 Uji Pengubahan dan Penghapusan Data Password.....                                | 20        |
| IV.6.1 Ubah Data.....                                                                 | 20        |
| IV.6.2 Hapus Data.....                                                                | 21        |
| IV.6.3 Pembaruan Hanya Pada Mode Normal.....                                          | 21        |
| IV.7 Uji Penyimpanan Vault di Server.....                                             | 22        |
| IV.7.1 Vault sebagai BLOB.....                                                        | 22        |
| IV.7.2 Tanpa Baris Plaintext Akun.....                                                | 23        |
| IV.7.3 Server Menolak Data Sensitif.....                                              | 23        |
| IV.8 Uji Mode Backup.....                                                             | 24        |
| IV.8.1 Simulasi Server Tidak Dapat Diakses.....                                       | 24        |
| IV.8.2 Membuka Vault dengan Kombinasi Local dan Recovery Share.....                   | 24        |
| IV.8.3 Backup Vault Lokal sebagai Sumber Vault.....                                   | 25        |
| IV.8.4 Data Password dapat Dilihat pada Mode Backup.....                              | 25        |
| IV.8.5 Pembatasan CRUD terhadap User pada Mode Backup.....                            | 26        |
| IV.9 Uji Kegagalan Pemulihan.....                                                     | 26        |
| IV.9.1 Recovery Share Salah.....                                                      | 26        |
| IV.9.2 Backup Vault Dimodifikasi.....                                                 | 27        |
| IV.9.3 Data Password Tidak Ditampilkan.....                                           | 27        |
| IV.10 Uji Kriptografi Visual (bonus).....                                             | 27        |
| IV.10.1 Pembuatan QR Code dari Recovery Share.....                                    | 27        |
| IV.10.2 Dua Gambar Share Hasil Pembagian QR Code.....                                 | 29        |
| IV.10.3 Hasil Penggabungan Kedua Gambar Share.....                                    | 29        |

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| IV.10.4 Validasi Recovery Code Hasil Penggabungan Kedua Gambar Share..... | 30        |
| <b>V. Kesimpulan.....</b>                                                 | <b>31</b> |
| <b>VI. Daftar Pustaka.....</b>                                            | <b>32</b> |
| <b>VII. Lampiran.....</b>                                                 | <b>32</b> |
| Lampiran 1. Pranala Repositori GitHub.....                                | 32        |
| Lampiran 2. Pranala Video Demo.....                                       | 33        |
| Lampiran 3. Pembagian Tugas Anggota Kelompok.....                         | 33        |

### Daftar Tabel

|                                                    |    |
|----------------------------------------------------|----|
| Tabel 3.1 Peran Client dan Server.....             | 4  |
| Tabel 3.2 Struktur Vault dan Entri Password.....   | 6  |
| Tabel 3.3 Penyimpanan Lokal Klien.....             | 7  |
| Tabel 3.4 Database Server.....                     | 7  |
| Tabel 3.5 Distribusi Lokasi Penyimpanan Share..... | 8  |
| Tabel 7.1 Pembagian Tugas Anggota Kelompok.....    | 33 |

### Daftar Gambar

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Gambar 3.1 Gambaran Umum Sistem.....                                    | 4  |
| Gambar 3.2 Alur Akses setiap Mode.....                                  | 5  |
| Gambar 3.3 Skema Kriptografi Visual dengan QR Code.....                 | 11 |
| Gambar 4.1 Hasil Uji Pembuatan Vault Baru.....                          | 12 |
| Gambar 4.2 Register membalas 201.....                                   | 12 |
| Gambar 4.3 Recovery Share.....                                          | 12 |
| Gambar 4.4 Server Share.....                                            | 12 |
| Gambar 4.5 Local Share.....                                             | 13 |
| Gambar 4.6 Server menyimpan Metadata.....                               | 13 |
| Gambar 4.7 Local Share Tidak Plain.....                                 | 14 |
| Gambar 4.8 Master Password Benar.....                                   | 14 |
| Gambar 4.9 Master Password Salah.....                                   | 14 |
| Gambar 4.10 Master Password Benar dan Fetch Sukses.....                 | 15 |
| Gambar 4.11 Output Perintah List.....                                   | 16 |
| Gambar 4.12 Server Share Asli.....                                      | 16 |
| Gambar 4.13 Merusak Server Share.....                                   | 16 |
| Gambar 4.14 Pengujian dengan Password Benar.....                        | 16 |
| Gambar 4.15 Server Share telah Dikembalikan.....                        | 16 |
| Gambar 4.16 Menambahkan Data Password dengan Input Manual.....          | 17 |
| Gambar 4.17 Menambahkan Data Password dengan Pembangkitan Otomatis..... | 18 |
| Gambar 4.18 Penambahan Data Password yang telah Tersimpan.....          | 18 |
| Gambar 4.19 Nonce Sebelum Save.....                                     | 18 |
| Gambar 4.20 Nonce Setelah Save.....                                     | 19 |
| Gambar 4.21 Nonce Identik dengan Backup pada Server.....                | 20 |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Gambar 4.22 List pada Backup dan Mode Normal Identik.....                         | 20 |
| Gambar 4.23 Ubah Data Berhasil.....                                               | 21 |
| Gambar 4.24 Hapus Data Berhasil.....                                              | 21 |
| Gambar 4.25 Pembaruan Dikirim dan Sukses pada Mode Normal.....                    | 22 |
| Gambar 4.26 Pembaruan Ditolak pada Mode Backup.....                               | 22 |
| Gambar 4.27 Vault sebagai BLOB.....                                               | 23 |
| Gambar 4.28 Server Menolak Data Sensitif.....                                     | 24 |
| Gambar 4.29 Mode Normal Tidak Berhasil karena Server Dimatikan.....               | 24 |
| Gambar 4.30 Vault Terbuka dengan Kombinasi Local Share dan Recovery Share.....    | 25 |
| Gambar 4.31 Data Vault Berhasil Ditampilkan dari File Lokal pada Mode Backup..... | 25 |
| Gambar 4.32 Data Password pada Mode Backup (Read-only).....                       | 26 |
| Gambar 4.33 Perubahan Data Ditolak pada Mode Backup.....                          | 26 |
| Gambar 4.34 Pengujian Recovery Share Format Salah.....                            | 26 |
| Gambar 4.36 Ciphertext Dimodifikasi.....                                          | 27 |
| Gambar 4.38 QR Code Recovery Share.....                                           | 28 |
| Gambar 4.39 Hasil Scan QR Code Recovery Share.....                                | 28 |
| Gambar 4.40 Recovery Key di CLI sebagai Fallback Opsional.....                    | 29 |
| Gambar 4.41 Pembuatan Dua Visual Share oleh Program.....                          | 29 |
| Gambar 4.42 Visual Share 1.....                                                   | 29 |
| Gambar 4.43 Visual Share 2.....                                                   | 29 |
| Gambar 4.44 Hasil Penggabungan Kedua Visual Share.....                            | 30 |
| Gambar 4.45 Hasil Scan QR Code dari Penggabungan Dua Visual Share.....            | 31 |

## I. Pendahuluan

Hampir setiap layanan digital saat ini mengharuskan pengguna memiliki kata sandi, dan semakin banyaknya layanan yang digunakan membuat pengelolaan kata sandi menjadi tantangan tersendiri. Kebanyakan pengguna akhirnya memakai kata sandi yang sama di banyak tempat, atau memilih kata sandi yang mudah ditebak, sehingga jika satu akun berhasil dibobol, akun lainnya ikut terancam. Aplikasi pengelola kata sandi hadir sebagai solusi, namun sebagian besar masih mengandalkan satu server terpusat untuk menyimpan seluruh data, yang berarti jika server tersebut diretas atau mengalami gangguan, seluruh data pengguna bisa hilang atau bocor sekaligus. Laporan ini membahas implementasi sistem pengelola kata sandi terdistribusi yang menggabungkan enkripsi AES-128-GCM untuk melindungi isi vault dengan skema Shamir's Secret Sharing untuk memecah kunci enkripsi ke beberapa lokasi berbeda, sehingga tidak ada satu titik pun yang jika berhasil dikompromikan akan langsung membuka seluruh data pengguna.

## II. Dasar Teori

### II.1 Shamir Secret Sharing

*Shamir's Secret Sharing* adalah skema kriptografi yang dikembangkan oleh Adi Shamir pada tahun 1979, yang memungkinkan sebuah rahasia dipecah menjadi beberapa bagian yang disebut *share*, di mana hanya kombinasi sejumlah minimum *share* tertentu yang bisa mengembalikan rahasia aslinya. Skema ini dinyatakan dengan parameter  $(k, n)$ , di mana  $n$  adalah total *share* yang dibuat dan  $k$  adalah jumlah minimum *share* yang diperlukan untuk rekonstruksi. Cara kerjanya didasarkan pada sifat polinomial, yaitu bahwa sebuah polinomial berderajat  $k-1$  ditentukan secara unik oleh tepat  $k$  titik koordinat. Rahasia  $S$  disisipkan sebagai koefisien konstan polinomial atau  $f(0) = S$ , sementara setiap *share* merupakan titik koordinat  $(x, f(x))$  yang dievaluasi pada nilai  $x$  yang berbeda-beda. Karena sejumlah *share* yang kurang dari  $k$  tidak memberikan informasi apa pun tentang rahasia aslinya, skema ini menjamin kerahasiaan sempurna secara teori informasi. Sistem yang dibangun menggunakan skema  $(2, 3)$ , sehingga kunci enkripsi dibagi menjadi 3 *share* yang disimpan di tiga tempat berbeda dan cukup 2 *share* untuk memulihkan kunci tersebut.

### II.2 Interpolasi Lagrange

Interpolasi Lagrange adalah metode matematika untuk merekonstruksi atau mengevaluasi sebuah polinomial berdasarkan sekumpulan titik koordinat yang diketahui. Dalam implementasi *Shamir's Secret Sharing*, metode ini digunakan untuk menghitung nilai rahasia dengan cara mengevaluasi polinomial di titik  $x = 0$  menggunakan  $k$  titik koordinat yang dikumpulkan dari *share* yang tersedia. Rumus rekonstruksinya adalah  $S = f(0) = \sum y_i \cdot L_i(0) \bmod p$ , di mana  $L_i(0)$  adalah basis polinomial Lagrange yang dihitung dari koordinat  $x$  seluruh *share* yang terlibat. Seluruh operasinya dilakukan dalam aritmatika modular di atas *finite field* sehingga

hasilnya selalu berupa bilangan bulat yang terdefinisi tanpa melibatkan bilangan pecahan. Keunggulan metode ini adalah nilai polinomial di titik tertentu bisa langsung dihitung tanpa perlu menyelesaikan sistem persamaan linier untuk mencari semua koefisiennya, sehingga rekonstruksi kunci bisa dilakukan dengan cepat di sisi klien.

### II.3 Enkripsi Simetris AES-128-GCM

AES (Advanced Encryption Standard) adalah algoritma enkripsi simetris yang ditetapkan sebagai standar internasional oleh NIST pada tahun 2001 dan sampai sekarang masih menjadi salah satu algoritma enkripsi paling banyak digunakan. Disebut simetris karena kunci yang dipakai untuk mengenkripsi dan mendekripsi data adalah kunci yang sama. Versi yang digunakan dalam sistem ini adalah AES-128, artinya panjang kuncinya 128 bit. Mode operasi yang dipilih adalah GCM (Galois/Counter Mode), yaitu mode yang tidak hanya mengenkripsi data tetapi sekaligus menghasilkan authentication tag sebagai jaminan integritas, sehingga jika data diubah oleh pihak lain setelah dienkripsi, proses dekripsi akan langsung gagal. Setiap operasi enkripsi juga memerlukan *nonce* sepanjang 96 bit yang dibangkitkan secara acak dan hanya digunakan sekali, sehingga enkripsi data yang sama dengan kunci yang sama pun akan menghasilkan *ciphertext* yang berbeda setiap kalinya.

### II.4 Key Derivation Function

Key Derivation Function (KDF) adalah fungsi yang mengubah sebuah kata sandi menjadi kunci kriptografi yang aman dan berukuran tetap sesuai kebutuhan algoritma enkripsi. Kata sandi yang dimasukkan pengguna umumnya terlalu pendek dan tidak cukup acak untuk langsung dijadikan kunci enkripsi, sehingga perlu melalui proses transformasi terlebih dahulu. Sistem ini menggunakan PBKDF2 (Password-Based Key Derivation Function 2) dengan fungsi hash SHA-256, yang cara kerjanya adalah mengaplikasikan HMAC-SHA256 secara berulang sebanyak 200.000 iterasi terhadap kombinasi kata sandi dan nilai *salt* yang dibangkitkan secara acak. *Salt* ditambahkan agar dua pengguna dengan kata sandi yang sama tetap menghasilkan kunci yang berbeda, sementara jumlah iterasi yang banyak memang sengaja dibuat agar prosesnya lambat, tujuannya memperlambat penyerang yang mencoba menebak kata sandi satu per satu secara otomatis.

### II.5 Pembangkit Bilangan Acak Kriptografis (CSPRNG)

CSPRNG (Cryptographically Secure Pseudo-Random Number Generator) adalah pembangkit bilangan acak yang dirancang khusus untuk kebutuhan kriptografi, di mana nilai yang dihasilkannya tidak bisa diprediksi secara komputasional meskipun penyerang mengetahui sebagian dari output sebelumnya. Ini berbeda dari fungsi *random* biasa dalam pemrograman umum yang menggunakan seed deterministik sehingga polanya bisa ditebak jika *seed*-nya diketahui. Dalam sistem ini, CSPRNG digunakan untuk membangkitkan *master key*, *nonce* pada setiap operasi AES-GCM, salt untuk proses derivasi kunci, serta menghasilkan kata sandi acak yang kuat saat

pengguna menggunakan fitur *generate password*. Implementasinya menggunakan modul *secrets* bawaan Python 3 yang mengambil entropi langsung dari sistem operasi.

## II.6 Visual Cryptography

Kriptografi visual adalah teknik kriptografi yang memecah sebuah gambar menjadi beberapa bagian yang masing-masing terlihat seperti kumpulan piksel acak, dan hanya ketika bagian-bagian tersebut ditumpuk atau digabungkan kembali barulah gambar aslinya terlihat. Teknik ini pertama kali diperkenalkan oleh Moni Naor dan Adi Shamir pada tahun 1994. Dalam implementasi pada sistem ini, gambar biner dipisahkan menjadi dua *share* menggunakan operasi XOR, di mana *share* pertama adalah piksel acak yang dibangkitkan oleh CSPRNG, sementara *share* kedua adalah hasil XOR antara *share* pertama dengan gambar aslinya. Teknik ini digunakan sebagai fitur tambahan untuk mengamankan QR Code *recovery share*, yaitu dengan memecahnya menjadi dua gambar terpisah yang masing-masing tidak mengungkapkan informasi apa pun, dan hanya ketika keduanya di-XOR kembali barulah QR Code aslinya muncul.

## II.7 QR Code

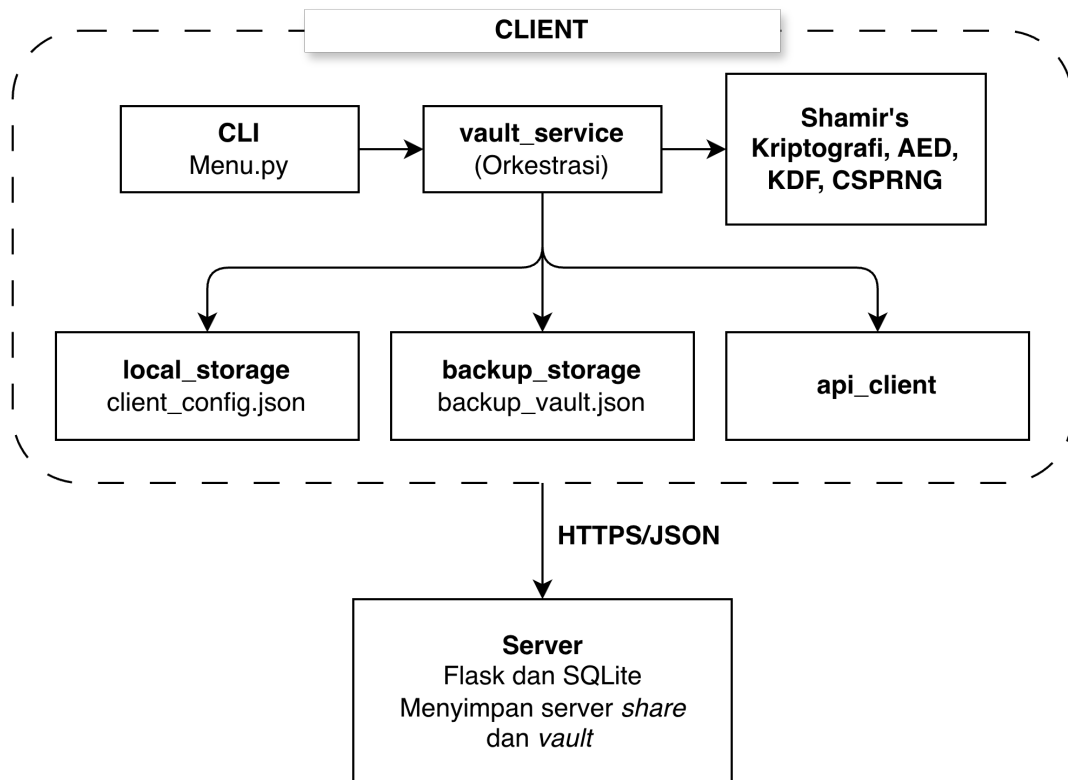
QR Code (*Quick Response Code*) adalah jenis kode dua dimensi yang dapat menyimpan data dalam bentuk pola kotak hitam-putih dan dibaca menggunakan kamera atau pemindai. Berbeda dari barcode satu dimensi yang hanya mampu menyimpan angka dalam jumlah terbatas, QR Code bisa menyimpan ratusan hingga ribuan karakter termasuk huruf, angka, dan karakter khusus dalam satu gambar kecil. Dalam sistem ini, QR Code dimanfaatkan sebagai media untuk menyimpan *recovery share* kepada pengguna, sehingga pengguna tidak perlu menyalin *string* panjang secara manual dan cukup menyimpan atau mencetak gambarnya saja. Untuk menambah lapisan keamanan, QR Code *recovery share* ini juga bisa diproses lebih lanjut menggunakan kriptografi visual sehingga dipecah menjadi dua gambar terpisah yang harus digabungkan kembali saat dibutuhkan.

# III. Perancangan Sistem

## III.1 Gambaran Umum Sistem

Sistem yang dibangun adalah pengelola kata sandi terdistribusi berbasis CLI yang menggabungkan enkripsi AES-128-GCM untuk melindungi data dengan skema Shamir's Secret Sharing (2,3) untuk mendistribusikan kunci enkripsi. Ide dasarnya adalah memastikan tidak ada satu entitas pun yang menyimpan cukup informasi untuk membuka vault secara mandiri. Kunci enkripsi utama (master key) dipecah menjadi tiga *share* yang disimpan di tiga lokasi berbeda, yaitu perangkat pengguna, server, dan *recovery share* yang dipegang pengguna sendiri. Untuk membuka vault dibutuhkan minimal dua *share*, sehingga jika server diretas atau perangkat hilang, data tetap

aman. Seluruh operasi kriptografi dilakukan di sisi client sehingga server tidak pernah mengetahui isi data maupun kunci enkripsinya.



**Gambar 3.1** Gambaran Umum Sistem

## III.2 Arsitektur Sistem

### III.2.1 Peran *Client* dan Server

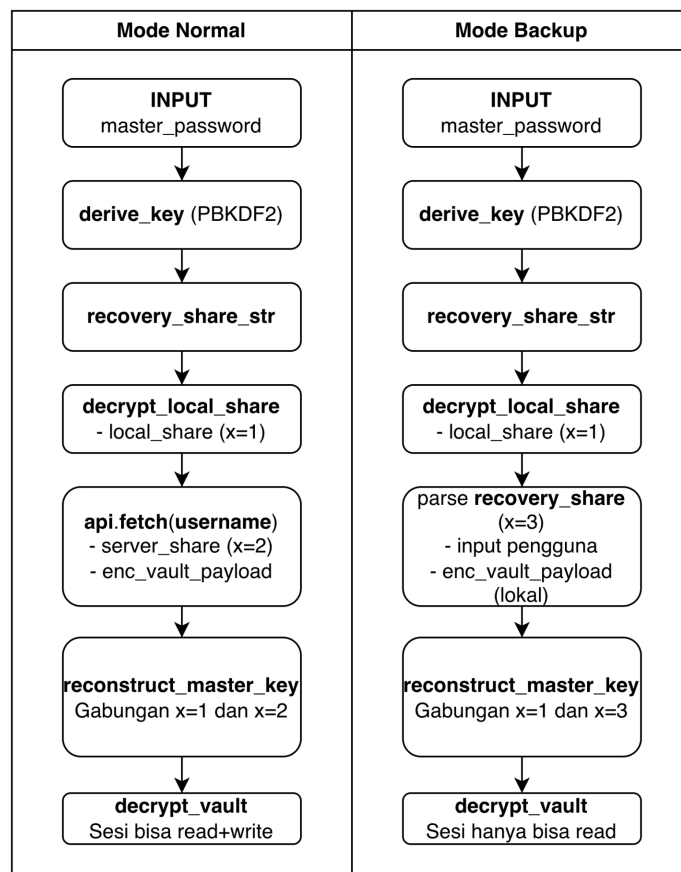
*Client* adalah satu-satunya lingkungan terpercaya dalam sistem ini. Semua proses kriptografi, mulai dari pembangkitan kunci, enkripsi, dekripsi, hingga rekonstruksi *share*, sepenuhnya terjadi di *client*. *Server* hanya menyimpan dua hal, yaitu *server share* ( $\text{share } x=2$ ) dan *vault* yang sudah dienkripsi oleh *client*. Dengan desain ini, bahkan jika *server* dibobol, penyerang hanya mendapat satu dari tiga *share* yang dibutuhkan sehingga tidak bisa membuka *vault*. Komponen *client* terdiri dari beberapa lapisan, *file* yang terlibat dan tanggung jawabnya dirangkum dalam tabel berikut.

**Tabel 3.1** Peran *Client* dan Server

| File                             | Lapisan   | Tanggung Jawab                                                 |
|----------------------------------|-----------|----------------------------------------------------------------|
| client/cli/menu.py               | Antarmuka | Tampilan menu, input pengguna, loop sesi interaktif            |
| client/services/vault_service.py | Logika    | Orkestrasi seluruh alur: setup vault, buka vault, CRUD, simpan |

|                                               |             |                                                  |
|-----------------------------------------------|-------------|--------------------------------------------------|
| <code>client/services/api_client.py</code>    | Jaringan    | Semua komunikasi HTTP ke server                  |
| <code>client/storage/local_storage.py</code>  | Penyimpanan | Baca/tulis data/ <code>client_config.json</code> |
| <code>client/storage/backup_storage.py</code> |             | Baca/tulis data/ <code>backup_vault.json</code>  |
| <code>client/models/password_entry.py</code>  | Model       | Definisi struktur data entri password            |
| <code>client/crypto/</code>                   | Kriptografi | Semua modul kriptografi                          |

### III.2.2 Alur Akses Mode Normal dan Mode *Backup*



**Gambar 3.2** Alur Akses setiap Mode

Perbedaan utama antara dua mode ini selain sumber *share*-nya adalah hak akses operasinya. Mode normal mengizinkan seluruh operasi termasuk penambahan, pengubahan, dan penghapusan data karena perubahan bisa disinkronkan ke *server*. Mode *backup* bersifat *read-only* karena *vault* dibuka dari salinan lokal dan tidak ada jalur untuk menyimpan perubahan kembali ke *server*.

### III.3 Rancangan Database

Penyimpanan data dirancang terpisah antara klien dan server, di mana masing-masing komponen hanya menyimpan data yang benar-benar diperlukan untuk perannya. Klien menyimpan data konfigurasi dan cadangan secara lokal dalam berkas JSON, sedangkan server menyimpan *vault* terenkripsi beserta *share* dan metadata di dalam basis data SQLite.

#### III.3.1 Struktur Vault dan Entri Password

*Vault* adalah kumpulan seluruh data akun pengguna yang disimpan sebagai satu kesatuan. Secara logis *vault* berbentuk array JSON yang setiap elemennya merepresentasikan satu entri password. Struktur tiap entri didefinisikan pada `client/models/password_entry.py` melalui `dataclass PasswordEntry` dengan atribut sebagai berikut.

**Tabel 3.2** Struktur *Vault* dan Entri Password

| Atribut                   | Tipe          | Keterangan                                          |
|---------------------------|---------------|-----------------------------------------------------|
| <code>id</code>           | string (UUID) | Pengenal unik, dibangkitkan otomatis tiap entri     |
| <code>nama_layanan</code> | string        | Nama layanan, wajib                                 |
| <code>username</code>     | string        | Username/email akun, wajib                          |
| <code>password</code>     | string        | Password tersimpan, wajib                           |
| <code>catatan</code>      | string        | Catatan tambahan, opsional ( <i>default</i> kosong) |

Sebelum disimpan, seluruh daftar entri di serialisasi menjadi satu string JSON oleh `client/services/vault_service.py`, lalu string tersebut dienkripsi sebagai satu blok utuh menggunakan AES-128-GCM. Dengan pendekatan mengenkripsi seluruh *vault* sebagai satu kesatuan, baik server maupun berkas *backup* tidak pernah melihat batas antara entri, yang tersimpan hanyalah satu *ciphertext*. Penambahan, pengubahan, dan penghapusan entri dilakukan pada struktur di memori sisi klien, kemudian seluruh *vault* dienkripsi ulang dari awal dengan *nonce* baru.

#### III.3.2 Penyimpanan Lokal Klien

Klien menyimpan dua berkas berformat JSON di dalam folder `data/`. Berkas pertama, `client_config.json`, ditulis oleh `client/storage/local_storage.py` saat *vault* pertama kali dibuat, dengan isi sebagai berikut:

**Tabel 3.3** Penyimpanan Lokal Klien

| Atribut           | Keterangan                                                            |
|-------------------|-----------------------------------------------------------------------|
| username          | Identitas pengguna untuk akses ke server                              |
| enc_local_payload | <i>Local share</i> terenkripsi, berupa objek {nonce, ciphertext, tag} |
| kdf_salt          | <i>Salt</i> PBKDF2 dalam base64                                       |
| iterations        | Jumlah iterasi KDF (200.000)                                          |

### III.3.3 Database Server

Server menyimpan data pada basis data SQLite yang skemanya didefinisikan di `server/db/schema.sql` dan diinisiasi oleh `server/db/database.py`. Terdapat dua tabel:

**Tabel 3.4** Database Server

| Tabel  | Kolom           | Tipe           | Keterangan                                                       |
|--------|-----------------|----------------|------------------------------------------------------------------|
| users  | id              | INTEGER PK     | Pengenal pengguna                                                |
|        | username        | TEXT<br>UNIQUE | Identitas pengguna                                               |
|        | created_at      | TEXT           | Waktu pembuatan                                                  |
| vaults | id              | INTEGER PK     | Pengenal <i>vault</i>                                            |
|        | user_id         | INTEGER FK     | Relasi ke <code>users</code>                                     |
|        | encrypted_vault | BLOB           | <i>Vault</i> terenkripsi ( <i>byte</i> mentah)                   |
|        | vault_nonce     | BLOB           | <i>Nonce</i> AES-GCM                                             |
|        | vault_tag       | BLOB           | <i>Authentication tag</i> AES-GCM                                |
|        | server_share    | TEXT           | <i>Server share</i> dalam<br>JSON { "x" : 2,<br>"y" : "base64" } |
|        | updated_at      | TEXT           | Waktu pembaruan terakhir                                         |

Keputusan rancangan terpenting adalah menyimpan isi *vault* sebagai BLOB pada kolom `encrypted_vault`, bukan sebagai kolom-kolom *plaintext* seperti nama layanan atau password. Dengan begitu, dari sudut pandang server, *vault* hanyalah sekumpulan *byte* acak dan server tidak memiliki kolom apapun yang dapat mengungkapkan isi akun pengguna. *Nonce*

dan *authentication tag* disimpan karena dibutuhkan klien untuk mendeskripsi sekaligus memverifikasi integritas *vault*. *Server share* disimpan sebagai teks JSON karena nilainya tidak sensitif secara mandiri, satu *share* saja tidak cukup untuk membuka *vault*.

Operasi baca/tulis tabel ditangani oleh `server/models.py`. Saat *register* *term payload* terenkripsi dari klien di-*decode* dari base64 menjadi BLOB sebelum disimpan. Saat *fetch*, BLOB di-*encode* kembali ke base64 untuk dikirim ke klien. Saat *update*, hanya kolom *ciphertext*, *nonce*, *tag*, dan `update_at` yang diperbarui. Rancangan ini menjaga prinsip *zero-knowledge*, server tidak pernah menyimpan *master key* utuh, *local share*, *recovery share*, maupun isi *vault* dalam bentuk *plaintext*.

### III.4 Rancangan Distribusi Kunci

Seluruh logika distribusi kunci ada di `client/crypto/shamir.py`. *Master key* dibangkitkan sebagai 16 *byte* acak menggunakan CSPRNG melalui `generate_master_key()`, lalu dipecah menggunakan `split_master_key()` yang memanggil `Shamir.split(2, 3, master_key)` dari library `pycryptodome`. Fungsi ini menghasilkan tiga pasang koordinat (*x*, *y*) yang masing-masing merepresentasikan satu titik pada polinomial berderajat 1 yang konstanta bebasnya adalah *master key* itu sendiri. Setiap *share* direpresentasikan sebagai *dict* `{"x": int, "y": "base64"}` di mana nilai *y* selalu 16 *byte*. Distribusi ketiga *share* ke masing-masing lokasi penyimpanan dirangkum dalam tabel berikut.

**Tabel 3.5** Distribusi Lokasi Penyimpanan Share

| Share    | X | Lokasi                               | Bentuk Akhir                                        |
|----------|---|--------------------------------------|-----------------------------------------------------|
| Local    | 1 | <code>data/client_config.json</code> | Terenkripsi dengan kunci turunan password           |
| Server   | 2 | Database SQLite server               | JSON string, dikirim via <code>api_client.py</code> |
| Recovery | 3 | Dipegang pengguna                    | JSON string, ditampilkan sekali di CLI              |

*Recovery share* di-*serialize* menggunakan `serialize_share()` menjadi JSON string kompak seperti `{"x": 3, "y": "base64..."}` yang bisa disalin dan disimpan pengguna secara mandiri. *String* inilah yang diminta kembali saat mode *backup* diaktifkan.

### III.5 Rancangan Enkripsi

*Local share* tidak bisa disimpan mentah ke *disk* karena siapa pun yang mengakses *file* tersebut bisa langsung mendapatkan satu *share*, sehingga *local share* dienkripsi menggunakan kunci yang diturunkan dari *master password* pengguna melalui PBKDF2 HMAC SHA256 yang diimplementasikan di `client/crypto/kdf.py`. Proses derivasi kunci menggunakan algoritma hash SHA 256 karena merupakan standar yang tersedia di semua *platform*, panjang *salt* sebesar 16 bytes untuk mencegah *rainbow table attack*, panjang output *key* sebesar 16 bytes agar sesuai dengan kebutuhan AES 128, serta jumlah iterasi sebanyak 200.000 untuk memperlambat *brute force*. *Salt* dibangkitkan secara acak menggunakan `generate_salt()` saat *setup vault*, kemudian disimpan bersama *local share* terenkripsi di `client_config.json`, sehingga proses derivasi kunci dapat direproduksi saat *login* berikutnya dengan menggabungkan *salt* yang tersimpan dan *master password* yang dimasukkan pengguna.

*Vault* dienkripsi menggunakan `encrypt_vault()` di `client/crypto/aes_gcm.py`. Setiap pemanggilan fungsi ini membangkitkan *nonce* baru 12 byte secara acak menggunakan CSPRNG, sehingga *ciphertext vault* selalu berbeda bahkan jika isinya tidak berubah. Mode GCM menghasilkan *authentication tag* 16 byte yang memastikan dekripsi akan gagal jika data dimodifikasi, sehingga kesalahan *password* maupun kerusakan data terdeteksi secara otomatis tanpa pengecekan tambahan. Setiap kali pengguna menjalankan perintah *save*, *vault* dienkripsi ulang dari nol dengan *nonce* baru, hasilnya dikirim ke *server* via `api_client.py`, lalu file `backup_vault.json` juga diperbarui. Proses ini memastikan *server* dan *backup* lokal selalu sinkron dengan kondisi *vault* terbaru.

### III.6 Antarmuka CLI dan Integrasi

Antarmuka CLI dirancang tanpa menggunakan *framework* eksternal dan seluruhnya diimplementasikan di `client/cli/menu.py`, dengan fungsi fungsi yang dipanggil dari `client/main.py` sebagai *entry point*. Saat dijalankan, program menampilkan menu utama yang terdiri dari empat pilihan yaitu *setup vault* baru, buka *vault* mode normal, buka *vault* mode *backup*, dan keluar. Setelah *vault* terbuka, pengguna masuk ke sesi interaktif yang menerima beberapa perintah, yaitu *list* untuk menampilkan semua entri tanpa *password* yang tersedia pada mode normal dan mode *backup*, “Show” untuk menampilkan detail entri termasuk *password* pada mode normal dan mode *backup*, “Add” untuk menambah entri baru yang hanya tersedia pada mode normal, “Edit” untuk mengubah *field* entri tertentu yang hanya tersedia pada mode normal, “Delete” untuk menghapus entri dari *vault* yang hanya tersedia pada mode normal, “Save” untuk melakukan re-enkripsi dan sinkronisasi ke *server* yang hanya tersedia pada mode normal, serta “Exit” untuk keluar dari sesi yang tersedia pada mode normal dan mode *backup*. Saat mode *backup* aktif, perintah “Add”, “Edit”, “Delete”, dan “Save” ditolak langsung di layer CLI sebelum mencapai

*service* karena perubahan tidak bisa disinkronkan ke *server*. Saat pengguna mengetik “Exit” dalam kondisi masih ada perubahan yang belum disimpan, program secara otomatis menampilkan konfirmasi untuk menyimpan terlebih dahulu. Seluruh orkestrasi logika ditangani oleh `vault_service.py` yang berperan sebagai perantara antara CLI dan modul modul kriptografi serta penyimpanan, sehingga CLI tidak memanggil fungsi kriptografi secara langsung dan pemisahan antara tampilan dan logika tetap terjaga.

### III.7 Password Generator

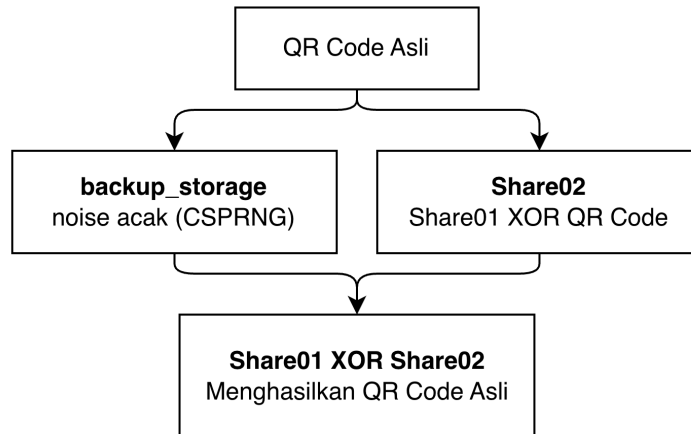
Fitur ini diimplementasikan di `client/crypto/password_generator.py` dan dapat diakses saat pengguna menambah atau mengedit entri. Pengguna memilih antara memasukkan password manual atau membangkitkan secara otomatis dengan menentukan panjang yang diinginkan, *default* sebanyak 16 karakter. Password dibangkitkan menggunakan `secrets.choice()` yang memilih karakter secara acak dari alfabet yang mencakup huruf kecil, huruf kapital, angka, dan karakter khusus, sehingga terdapat total 94 karakter yang tersedia. Penggunaan modul *secrets* memastikan setiap karakter dipilih dengan distribusi seragam dari sumber entropi sistem operasi.

### III.8 QR Code untuk Recovery Share

Karena *recovery share* berbentuk JSON string yang cukup panjang, sistem menyediakan fitur untuk mengkonversinya ke QR Code agar lebih mudah disimpan dan dipindai kembali. Implementasinya terletak di `client/crypto/qr_recovery.py` menggunakan library *qrcode*. Share divalidasi terlebih dahulu sebelum di-*serialize* ke JSON kompak lalu di-*encode* menjadi gambar QR Code yang disimpan ke *file* yang ditentukan pengguna.

### III.9 Kriptografi Visual untuk Recovery Share

Sebagai lapisan keamanan tambahan, *QR Code recovery share* bisa dipecah menjadi dua gambar menggunakan kriptografi visual yang diimplementasikan di `client/crypto/visual_crypto.py`. Prinsipnya adalah operasi XOR pada level piksel: *share* pertama adalah gambar *noise* acak yang dibangkitkan CSPRNG, sementara *share* kedua adalah hasil XOR antara *share* pertama dengan gambar QR Code asli. Masing-masing gambar terlihat seperti *noise* dan tidak mengungkapkan informasi apa pun tentang QR Code aslinya. QR Code hanya bisa dikembalikan dengan meng-XOR kedua *share* tersebut menggunakan fungsi `merge_visual_shares()`.



**Gambar 3.3** Skema Kriptografi Visual dengan QR Code

## IV. Pengujian Program dan Analisis Hasil

### IV.1 Skenario Pengujian

Sebelum mulai *testing*, pastikan:

1. Dependensi sudah di-install: `python -m pip install -r requirements.txt`
2. Server sudah jalan: `python -m server.app` di terminal terpisah
3. Database *fresh*: Hapus `data/server.db` jika ada data lama
4. Client config *fresh*: Hapus `data/client_config.json` dan `data/backup_vault.json`
5. Program dijalankan dengan: `python -m client.main` di terminal terpisah

### IV.2 Uji Pembuatan Vault

#### IV.2.1 Buat *Vault* Baru dengan *Master Password* Valid

Pada pengujian ini dilakukan pembuatan *vault* baru dengan input sebagai berikut:

- Username: test
- Master password: testpassword
- Konfirmasi password: testpassword

```
(.venv) (base) alicia@alicias-macbook-6 II4021-Kriptografi-Tugas-4 % python -m client.main

=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup - tanpa server)
4. Keluar
Pilih: 1

--- Setup Vault Baru ---
Username: test
Master password:
Konfirmasi password:
[*] Membuat vault...

=====
RECOVERY SHARE - SIMPAN SEKARANG, HANYA TAMPIL SEKALI
=====
{"x":3,"y":"XyPJHJi2YVmSBv1YGMiQQ==" }
=====

[+] Visual recovery share dibuat:
1. /Users/alicia/Documents/II4021-Kriptografi-Tugas-4/data/recovery_visual_share_1.png
2. /Users/alicia/Documents/II4021-Kriptografi-Tugas-4/data/recovery_visual_share_2.png
Simpan kedua file ini terpisah; mode backup membutuhkan keduanya.
[+] Vault berhasil dibuat!
```

Gambar 4.1 Hasil Uji Pembuatan *Vault* Baru

*Master key* dibangkitkan acak sepanjang 16 byte (128-bit) lalu dipecah menjadi tiga *share* berkoordinat  $x = \{1, 2, 3\}$ . Rekonstruksi dari sembarang dua *share* ( $x_1+x_2$ ,  $x_1+x_3$ ,  $x_2+x_3$ ) menghasilkan *master key* yang identik, membuktikan skema (2,3) berjalan benar. *Vault* kosong berhasil dienkripsi AES-128-GCM dan data awal tersimpan ke server (register membalas 201).

```
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [05/Jun/2026 20:32:53] "GET /ping HTTP/1.1" 200 -
127.0.0.1 - - [05/Jun/2026 20:32:54] "POST /vault/register HTTP/1.1" 201 -
```

Gambar 4.2 Register membalas 201

#### IV.2.2 Pembagian *Master Key* menjadi Tiga *Share*

##### 1. *Recovery Share* (ditampilkan JSON dengan $x = 3$ )

*Recovery share* ditampilkan oleh CLI ketika membuat *vault* baru.

```
=====
RECOVERY SHARE - SIMPAN SEKARANG, HANYA TAMPIL SEKALI
=====
{"x":3,"y":"XyPJHJi2YVmSBv1YGMiQQ==" }
=====
```

Gambar 4.3 *Recovery Share*

##### 2. *Server Share* ( $x = 2$ )

*Server share* dapat ditampilkan dari basis data server.

```
sqlite> select server_share from vaults;
{"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="}
```

Gambar 4.4 *Server Share*

##### 3. *Local Share* ( $x = 1$ )

*Local share* terdapat pada file `data/client_config.json` yang dienkripsi dengan AES-GCM, dan bukan *plaintext*.

```
data > {} client_config.json > ...
1  {
2    "username": "test",
3    "enc_local_payload": {
4      "nonce": "/EB1vx51prCrdaF6",
5      "ciphertext": "9jtk3/PI/g0A6qm11Nasi6WgAlAPCs7yamIHbg3BAYfLIKXyf/Q=",
6      "tag": "rxfi0+ex75DJ+wuTM3N6Rw=="
7    },
8    "kdf_salt": "jLX+/50icax+N2TnwY61pA==",
9    "iterations": 200000
10 }
```

Gambar 4.5 Local Share

### IV.2.3 Recovery Share Dalam Format Teks

*Recovery share* ditampilkan sebagai teks JSON `{"x":3, "y":"base64..."}` yang memuat koordinat dan nilai *share*. Pada [Gambar 4.3](#) ditampilkan *recovery share* sebagai berikut.

```
{"x":3, "y":"XyPJHJi2YVmSBv1YGMiQQQ=="}
```

### IV.2.4 Server Menyimpan Metadata

Pada gambar berikut, ditampilkan bahwa server hanya menyimpan *server share*, *vault* terenkripsi, *nonce vault*, dan metadata yang diperlukan.

```
sqlite> select * from vaults join users on users.id = vaults.user_id;
id user_id encrypted_vault vault_nonce vault_tag server_share updated_at id username created_at
1 1 n0 ipw00Xp %0Jbcu0=0zd0h1 {"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="} 2026-06-05T13:32:54.091467+00:00 1 test 2026-06-05T13:32:54.090602+00:00
sqlite>
```

Gambar 4.6 Server menyimpan Metadata

## IV.3 Uji Penyimpanan dan Perlindungan Local Share

### IV.3.1 Local Share Tidak Plain

Pada `client_config.json`, *local share* tersimpan sebagai `enc_local_payload` (`{nonce, ciphertext, tag}`), bukan *share* asli. Bahwa kuncinya diturunkan dari *master password* dibuktikan oleh tersimpannya `kdf_salt` dan `iterations` (parameter PBKDF2) di berkas yang sama, serta oleh kontras IV.3.2 dan IV.3.3. Hanya *master password* benar yang dapat membukanya.

```
"enc_local_payload": {
  "nonce": "/EB1vx51prCrdaF6",
  "ciphertext": "9jtk3/PI/g0A6qm11Nasi6WgAlAPCs7yamIHbg3BAYfLIKXyf/Q=",
  "tag": "rxfi0+ex75DJ+wuTM3N6Rw=="
},
```

Gambar 4.7 Local Share Tidak Plain

### IV.3.2 Master Password Benar

Dengan *master password* `testpassword`, *local share* berhasil didekripsi sehingga proses dapat berlanjut. Pengujian ini berfokus pada lolosnya gerbang *local share*, alur normal penuh akan dibahas pada [IV.4](#).

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 2
Master password:
[+] Vault terbuka – MODE NORMAL
Mode: NORMAL | ketik 'help' untuk perintah
vault> █
```

Gambar 4.8 Master Password Benar

### IV.3.3 Master Password Salah

Dengan *master password* selain `testpassword`, sistem menolak dengan pesan `[!] Master password salah.` Kegagalan ini terjadi di tahap `decrypt_local_share`, yaitu sebelum klien menghubungi server. Hal ini membuktikan *local share* benar-benar terlindungi oleh *master password*.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 2
Master password:
[!] Master password salah.
```

Gambar 4.9 Master Password Salah

## IV.4 Uji Akses Normal

### IV.4.1 Buka *Vault* dengan *Master Password* Benar (Kombinasi *Local* + *Server Share*)

Saat server aktif, klien mengambil *server share* dan *vault* terenkripsi via `GET /vault/fetch/<username>` (dibalas 200), lalu menggabungkannya dengan *local share*.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 2
Master password:
[+] Vault terbuka – MODE NORMAL
Mode: NORMAL | ketik 'help' untuk perintah

vault> 
```

```
127.0.0.1 - - [05/Jun/2026 23:37:05] "GET /vault/fetch/test HTTP/1.1" 200
```

Gambar 4.10 *Master Password* Benar dan *Fetch* Sukses

### IV.4.2 Rekonstruksi *Master Key* dari Dua *Share* Valid

Rekonstruksi (`reconstruct_master_key`) adalah langkah internal di klien. Keberhasilannya dibuktikan oleh terbukanya *vault* dan tampilnya isi (IV.4.3). Jika *master key* hasil rekonstruksi salah, dekripsi AES-GCM pasti gagal. (Hal ini dapat dilihat dari Gambar 4.10)

### IV.4.3 *Vault* Didekripsi dan Isi Ditampilkan

*Master key* hasil rekonstruksi mendekripsi *vault*, dan perintah `list` menampilkan isinya (*vault* kosong tampil `(vault kosong)`), yang juga menandakan dekripsi sukses).

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 2
Master password:
[+] Vault terbuka – MODE NORMAL
Mode: NORMAL | ketik 'help' untuk perintah

vault> list
(vault kosong)
vault> 
```

Gambar 4.11 Output Perintah List

#### IV.4.4 Master Password Salah atau Share Tidak Valid

Untuk melakukan pengujian ini, dilakukan perusakan `server_share` pada `database`, serta menggunakan mode normal dengan `password` yang benar `testpassword`. Langkah-langkah yang dilakukan sebagai berikut:

1. Catat dulu `server share` asli, agar bisa dikembalikan lagi

```
sqlite> SELECT server_share FROM vaults;
server_share
-----
{"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="}
```

Gambar 4.12 Server Share Asli

2. Rusak `server share`-nya (format tetap valid, nilainya yang salah)

```
sqlite> UPDATE vaults SET server_share='{"x":2,"y":"AAAAAAAAAAAAAAAAAAAAAA=="}';
sqlite> .quit
(.venv) (base) alicia@alicias-macbook-6 II4021-Kriptografi-Tugas-4 %
```

Gambar 4.13 Merusak Server Share

3. Jalankan CLI mode normal dengan password BENAR (`testpassword`)

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup - tanpa server)
4. Keluar
Pilih: 2
Master password:
[!] Dekripsi vault gagal. Master key tidak valid atau vault rusak.
```

Gambar 4.14 Pengujian dengan Password Benar

4. Kembalikan `server share` asli

```
sqlite> UPDATE vaults SET server_share='{"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="}'
sqlite>
sqlite> SELECT server_share FROM vaults;
{"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="}
```

Gambar 4.15 Server Share telah Dikembalikan

Dengan `share` yang tidak valid, rekonstruksi menghasilkan `master key` salah sehingga dekripsi `vault` gagal dengan pesan "Dekripsi vault gagal. Master key tidak valid atau vault rusak.", dan tidak ada data ditampilkan. Titik gagalnya berbeda dari [IV.3.3](#) (yang gagal lebih awal di `local share`).

## IV.5 Uji Penambahan Data Password

### IV.5.1 Menambahkan Data *Password* dengan Input Manual

Entri ditambahkan via `add` dengan *password* manual dan berhasil masuk ke *vault*.

```
vault> add
--- Tambah Entri ---
Nama layanan: Github
Username/email: test@gmail.com
Password: (1) Manual (2) Generate otomatis [1]: 1
Password:
Catatan (opsional): Akun utama
[+] 'Github' ditambahkan. Jangan lupa 'save'.
vault> save
[+] Vault disimpan dengan nonce baru.
vault> list
```

| No | Layanan | Username       |
|----|---------|----------------|
| 0  | Github  | test@gmail.com |

```
vault> show 0
```

Layanan : Github  
Username : test@gmail.com  
Password : testtestpassword  
Catatan : Akun utama

Gambar 4.16 Menambahkan Data *Password* dengan Input Manual

### IV.5.2 Menambahkan Data *Password* dengan Pembangkitan Otomatis menggunakan CSPRNG

```
vault> add
--- Tambah Entri ---
Nama layanan: Instagram
Username/email: test.insta
Password: (1) Manual (2) Generate otomatis [1]: 2
Panjang [16]: 12
Password dibangkitkan: <C_Aw|24~^,~
Catatan (opsional): Main Account
[+] 'Instagram' ditambahkan. Jangan lupa 'save'.
vault> save
[+] Vault disimpan dengan nonce baru.
vault> list
```

| No | Layanan   | Username       |
|----|-----------|----------------|
| 0  | Github    | test@gmail.com |
| 1  | Instagram | test.insta     |

```
vault> show 1
```

Layanan : Instagram  
Username : test.insta  
Password : <C\_Aw|24~^,~  
Catatan : Main Account

**Gambar 4.17** Menambahkan Data *Password* dengan Pembangkitan Otomatis

### IV.5.3 Enkripsi Ulang dengan *Nonce* Baru

Setelah *save*, *vault* dienkripsi ulang. *Nonce vault* di server berubah dibanding sebelumnya, dan CLI menampilkan **[+] Vault disimpan dengan nonce baru.**

```
vault> add
--- Tambah Entri ---
Nama layanan: Tiktok
Username/email: test.tiktok
Password: (1) Manual (2) Generate otomatis [1]: 2
Panjang [16]: 10
Password dibangkitkan: 0I[r!*'.(r
Catatan (opsional):
[+] 'Tiktok' ditambahkan. Jangan lupa 'save'.
vault> save
[+] Vault disimpan dengan nonce baru.
vault> 
```

**Gambar 4.18** Penambahan Data *Password* yang telah Tersimpan

*Nonce vault* sebelum *save* adalah **&7B(FH0i**

```
sqlite> sqlite3 select * from vaults;
id user_id encrypted_vault vault_nonce vault_tag server_share updated_at
1 1 X00Bo 67B(FH0i 0. 060n0 {"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg="} 2026-06-06T08:01:30.201984+00:00
4040.r-0
0007">0
010
00
0
/NU3\cd0 0000R
[ftq00
@0eRH0f0"#(
0 P<0.00-N
I0;00x00.00S(r0kc0n00
:0n30z
000Yk0c+
P0>00
0
@U0+0+0A-pl
m0000 ]0
00x0
0
0/90
0L0
0S0H;[]0000K
00;0;00h]0.
0000 00V
m
00<0/+00<00000Kf0
00F0RL\|0=0~0!{Y00
A]000H0.00007*0p00n0470n100Q10A000X000]
```

**Gambar 4.19** *Nonce* Sebelum *Save*

*Nonce vault* setelah *save* berubah menjadi **ãOI0rC π**

```
sqlite> select * from vaults;
```

| id | user_id | encrypted_vault                                                                                                                                                                                                                                                                                                                                                                    | vault_nonce | vault_tag          | server_share                           | updated_at                       |
|----|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|--------------------|----------------------------------------|----------------------------------|
| 1  | 1       | W0[C0S0W0bb*0+0/<br>00S00<br>0+01=I000C000R0}a0<br>00W<br>y0}<br>0≡ 8A0<br><br>00000i<br><br>E0k0w0000cS0<br>0(0),Y00<br><br>0,I m00,o0ij0<br>000V<br>000 F0000<br><br>0p<br>+0/50G000G0[<rh00V0jh0'<br>0<br>000<br>000i000o01k`"k20X/Y00];0<br>{00<br>00<N<br>Mc\0700>F┘<br><br>00008j0I0J0.<br>02Q^0%<br>~0:~P<br>(=2ik00H#rd<br>700Ju<br>1YZ00+0j060g00=w0d0c00J0<br>>00E0<br>D | ra0I0rC π   | I0009{J000p<br>.0" | {"x":2,"y":"dd34A2PB8ofvXtZtfIw7eg=="} | 2026-06-06T09:05:19.638562+00:00 |

Gambar 4.20 Nonce Setelah Save

#### IV.5.4 Backup Lokal Diperbarui

Setelah *save*, *nonce* pada `backup_vault.json` sama dengan *nonce* di server, menandakan *backup* lokal sinkron. Berikut merupakan file `backup_vault.json` dengan *nonce* "3IphT0nxckMJ0rUF".

```
{
  "enc_vault_payload": {
    "nonce": "3IphT0nxckMJ0rUF",
    "ciphertext": "V8BbQ9Am1JzgYt+VKsQqti8Km5Mk+ewT1SruuDE9SdzNMEOEi8JSvX1h+Na1E+zSVxZ5zH0gAtbrirk4QaYHCAjy6/rAvmkYCKWxa4RXkJu8q8SHU+Ya0Ci2XSxZ0/QZB43ZgCxJCW21z9mIb4BpaoUcuoTTVhGLneggRrqpSj0bEPhwHiv8LzW/R/eHxUeWWzxyaIHVvr9KaPwnEKsI/PvDHk+7vGnJ08ZvTzFrYcJrMuZYl1mMw1072AF7rPcZiJY8TgZNY1yUP8bn3olG3KUBGf6B5zhqmTHaxKr2SsrZoAffMlFelCUHfr3UiinQEig9Mmlr1vxII2QWN6zbSnUNMVla3fUqqWr5No5npO2fPVeeZLBkg9tK6gE+4JZftBVEAPE8chUTu8CjfyV9CN7L8YvQtoyKsbogOAJyBoAWJD9o1tFsLqGVs5RYd6MFC4zRY8DoQSHI6nxvLP81Fv81cM2G4hy6/LmfkjS5BDuIjqEsOwI2FK5MArdvrPEX2NXDLTqO+dZ9DPfobv+aghegkvtcOaC9koVf+oXJ8I/oEF8S3vhQP92u/PtdfBP/xzgH558qpOirZfIE7fikXmUGnqSORYZEhDe9uJ0Y",
    "tag": "SaSU8Tl7SlH7jnAX2avRIg=="
  }
}
```

Yang juga identik dengan *nonce* pada server yaitu "3IphT0nxckMJ0rUF".

```
(.venv) (base) alicia@alicias-macbook-6 II4021-Kriptografi-Tugas-4 % curl -s http://127.0.0.1:5000/vault/fetch/test | python3 -m json.tool
{
  "enc_vault_payload": {
    "ciphertext": "V8BbQ9Am1JzgYt+VKsQqt18Km5Mk+ewT1SruuDE9SdzNME0Ei8JSvX1h+Na1E+zSVxZ5zH0gAtbrirk40aYHCAjy6/rAvmkYCKWxa4RXkJu8q8SHU+Ya0Ci2XSxZ0/QZB43ZgCxJCW21z9mIb4BpaoUcuoTTVhGLneggRrqsJ0bEPHwHiv8LzW/R/eHxUeWwzxyaIHVVr9KaPwnEKsI/PvDHk+7vGnJ08ZvTzFrYCjrMuZYL1mMwL072AF7rPcZiJY8TgZNY1yUP8bn3oLG3KUBGf6B5zhqmTHaxKr2SsrZoAffmLFelCUHfr3UiiNQEIg9Mmlr1vxII2QWN6zbSnUNMVL3fUqqWr5No5np02fPVeeZLBkg9tK6gE+4JZFtBVEAPE8chUTu8CjfyV9CN7L8YvQtoyKsbog0AJyBoAWJD9o1tFsLqGVs5RYd6MFC4zRY8DoQSHI6nxvLP81Fv8lcM2G4hy6/LmfkjS5BDuIjqEs0wI2FK5MArdvrPEX2NXDLTq0+dZ9DPfobv+aghegkvct0aC9koVf+oXJ8I/oEF8S3vhQP92u/PtdfBP/xzgh558qp0irZfIE7fikXmUGnqS0RYZEhDe9uJ0Y",
    "nonce": "3IphT0nxckMJ0rUF",
    "tag": "SaSU8TL7SLH7jnAX2avRIg=="
  },
  "server_share": {
    "x": 2,
    "y": "dd34A2PB8ofvXtZtfIw7eg=="
  }
}
```

**Gambar 4.21** Nonce Identik dengan Backup pada Server

Pada CLI pun, membuktikan bahwa *backup vault* lokal juga diperbarui setelah perubahan pada mode normal.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup - tanpa server)
4. Keluar
Pilih: 3
Master password:
Recovery source: (1) visual share PNG (2) teks manual [1]: 2
Recovery share: {"x":3,"y":"XyPJHJi2YVmSBv1YGMiQQ==" }
[+] Vault terbuka - MODE BACKUP (read-only)
Mode: BACKUP (read-only) | ketik 'help' untuk perintah

vault> list
```

| No | Layanan   | Username       |
|----|-----------|----------------|
| 0  | Github    | test@gmail.com |
| 1  | Instagram | test insta     |
| 2  | Tiktok    | test.tiktok    |

**Gambar 4.22** List pada Backup dan Mode Normal Identik

## IV.6 Uji Pengubahan dan Penghapusan Data Password

### IV.6.1 Ubah Data

Setelah *edit* dan *save*, *password* yang diubah tetap bertahan saat *vault* dibuka kembali.

```

vault> show 0

Layanan : Github
Username : test@gmail.com
Password : testptestpassword
Catatan : Akun utama

vault> edit
Usage: edit <nomor>
vault> show 0

Layanan : Github
Username : test@gmail.com
Password : testptestpassword
Catatan : Akun utama

vault> edit 0
--- Edit: Github (Enter = tidak diubah) ---
Nama layanan [Github]:
Username [test@gmail.com]:
Ganti password? (y/n): y
(1) Manual (2) Generate [1]: 1
Password baru:
Catatan [Akun utama]: Sudah ganti pasword
[+] Diperbarui. Jangan lupa 'save'.
vault> save
[+] Vault disimpan dengan nonce baru.
vault> show 0

Layanan : Github
Username : test@gmail.com
Password : testpassword
Catatan : Sudah ganti password

```

**Gambar 4.23** Ubah Data Berhasil

## IV.6.2 Hapus Data

Setelah *delete* dan *save*, entri yang dihapus tidak lagi muncul saat *vault* dibuka ulang.

```

vault> delete 0
Hapus 'Github'? (y/n): y
[+] Dihapus. Jangan lupa 'save'.
vault> save
[+] Vault disimpan dengan nonce baru.
vault> list

```

| No | Layanan   | Username    |
|----|-----------|-------------|
| 0  | Instagram | test.insta  |
| 1  | Tiktok    | test.tiktok |

**Gambar 4.24** Hapus Data Berhasil

## IV.6.3 Pembaruan Hanya Pada Mode Normal

Pembaruan dikirim melalui `PUT /vault/update` yang hanya dapat dipanggil pada mode normal.

```
127.0.0.1 - - [06/Jun/2026 17:29:04] "PUT /vault/update HTTP/1.1" 200 -  
127.0.0.1 - - [06/Jun/2026 17:32:35] "PUT /vault/update HTTP/1.1" 200 -
```

**Gambar 4.25** Pembaruan Dikirim dan Sukses pada Mode Normal

Namun, pada mode *backup* pembaruan ditolak.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===  
1. Setup Vault Baru  
2. Buka Vault (Mode Normal)  
3. Buka Vault (Mode Backup – tanpa server)  
4. Keluar  
Pilih: 3  
Master password:  
Recovery source: (1) visual share PNG (2) teks manual [1]: 2  
Recovery share: {"x":3,"y":"XyPJHJi2YVmSBv1YGMiQQQ=="}  
[+] Vault terbuka – MODE BACKUP (read-only)  
Mode: BACKUP (read-only) | ketik 'help' untuk perintah  
  
vault> show 0  
  
Layanan : Instagram  
Username : test.insta  
Password : <C_Aw|24~^,~  
Catatan : Main Account  
  
vault> edit 0  
[!] Mode backup: tidak bisa edit.
```

**Gambar 4.26** Pembaruan Ditolak pada Mode *Backup*

## IV.7 Uji Penyimpanan Vault di Server

### IV.7.1 *Vault* sebagai BLOB

Kolom `encrypted_vault` bertipe BLOB (*byte mentah*) di SQLite.

```

sqlite> .schema vaults
CREATE TABLE vaults (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  encrypted_vault BLOB NOT NULL,
  vault_nonce BLOB NOT NULL,
  vault_tag BLOB NOT NULL,
  server_share TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
sqlite> select encrypted_vault from vaults;
encrypted_vault
-----
030
00dc000

06\00
03G0y
)0
000
000d0s00k=0zdz<0GQ|0[l0`;60%10Y0.0fx0000`0D000='0w00#N0i
4PE0h40p00!n0
0P
0000&000N+@

0bD000s0(0000>00000Dt0{0@_b0(
{Y0`Fe-G0{sZ0AN 0;003,gM
0E00j0

0$L0p0A0y5a
0
D^"?0F
0<000X0K
0z
0:Zw000
V070h'
|ä00D0I000u0SZ0|0{

00R0
0{000o0=_

:06;7B0~0P^00000?{>0
sqlite>

```

Gambar 4.27 Vault sebagai BLOB

#### IV.7.2 Tanpa Baris *Plaintext* Akun

Tabel `vaults` tidak memiliki kolom `nama_layanan`, `username`, `password`, atau `catatan`. Isi akun hanya ada di dalam *ciphertext*. (Lihat [Gambar 4.19](#) atau [Gambar 4.27](#)).

#### IV.7.3 Server Menolak Data Sensitif

Permintaan `register` yang menyertakan *field* terlarang seperti `master_key` ditolak status 400 (`request contains forbidden or unknown fields`). Hal ini membuktikan bahwa server tidak dapat menerima *master key*, *local share*, *recovery share*, maupun isi *vault* dalam bentuk *plaintext*.

```
(.venv) (base) alicia@alicias-macbook-6 II4021-Kriptografi-Tugas-4 % curl -i -X POST http://127.0.0.1:5000/vault/register \
-H "Content-Type: application/json" \
-d '{"username":"hacker","master_key":"seharusnya_ditolak"}'
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.13.2
Date: Sat, 06 Jun 2026 11:05:39 GMT
Content-Type: application/json
Content-Length: 57
Connection: close

{"error":"request contains forbidden or unknown fields"}
(.venv) (base) alicia@alicias-macbook-6 II4021-Kriptografi-Tugas-4 %
```

**Gambar 4.28** Server Menolak Data Sensitif

## IV.8 Uji Mode *Backup*

### IV.8.1 Simulasi Server Tidak Dapat Diakses

Setelah melakukan setup vault awal, matikan *server* dengan memasukkan input “CTRL+C” di terminal *server*. Alhasil, membuka *vault* dengan mode normal tidak akan berhasil.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 2
[!] Server tidak bisa diakses.
Coba mode backup? (y/n):
```

**Gambar 4.29** Mode Normal Tidak Berhasil karena Server Dimatikan

Ketika server tidak bisa diakses, akan muncul pilihan untuk melanjutkan mode *backup*.

### IV.8.2 Membuka *Vault* dengan Kombinasi *Local* dan *Recovery Share*

*Vault* berhasil dibuka menggunakan kombinasi *local share* (x=1) dan *recovery share* (x=3) tanpa perlu *server*.

```
[+] Vault terbuka – MODE BACKUP (read-only)
Mode: BACKUP (read-only) | ketik 'help' untuk perintah

vault> help

list          → tampilkan semua entri
show <no>     → detail entri + password
add           → tambah entri baru
edit <no>     → edit entri
delete <no>   → hapus entri
save          → enkripsi ulang dan simpan ke server
exit          → tutup vault

[Mode backup aktif: hanya list dan show]

vault> █
```

**Gambar 4.30** Vault Terbuka dengan Kombinasi *Local Share* dan *Recovery Share*

Dalam file `vault_service.py`, terdapat baris kode untuk merekonstruksi ulang *master key* tanpa `server_share`.

```
master_key = reconstruct_master_key([local_share,
recovery_share])
```

### IV.8.3 Backup Vault Lokal sebagai Sumber Vault

*Vault* yang ditampilkan adalah salinan lokal yang sudah di-*backup* saat *setup* awal. Jika ada data yang sudah ditambahkan sebelumnya, data tersebut akan ditampilkan. *Vault* data di-load dari *backup* lokal, *decrypt* dengan `master_key` yang sudah di-*reconstruct* sehingga tidak ada *API call* ke *server*.

```
[!] Server tidak bisa diakses.
Coba mode backup? (y/n): y
Master password:
Recovery source: (1) visual share PNG (2) teks manual [1]: 1
Visual share 1 [/Users/amandarizki/Desktop/Semester 06/Kriptografi/II4021-Kriptografi-Tugas-4/data/recovery_visual_share_1.png]:
Visual share 2 [/Users/amandarizki/Desktop/Semester 06/Kriptografi/II4021-Kriptografi-Tugas-4/data/recovery_visual_share_2.png]:
[+] Vault terbuka – MODE BACKUP (read-only)
Mode: BACKUP (read-only) | ketik 'help' untuk perintah

vault> help

list          → tampilkan semua entri
show <no>     → detail entri + password
add           → tambah entri baru
edit <no>     → edit entri
delete <no>   → hapus entri
save          → enkripsi ulang dan simpan ke server
exit          → tutup vault

[Mode backup aktif: hanya list dan show]

vault> list

No  Layanan                Username
-----
0   email                  sambo@email.com
```

**Gambar 4.31** Data *Vault* Berhasil Ditampilkan dari File Lokal pada Mode Backup

### IV.8.4 Data Password dapat Dilihat pada Mode Backup

User dapat membaca semua data *password* dalam mode *backup* melalui perintah “show” pada menu Vault.

```
vault> list

No      Layanan      Username
-----
0       email         sambo@email.com

vault> show 0

Layanan   : email
Username  : sambo@email.com
Password  : >2AMP_yf
Catatan   : -
```

Gambar 4.32 Data Password pada Mode Backup (Read-only)

#### IV.8.5 Pembatasan CRUD terhadap User pada Mode Backup

Semua operasi oleh *user* yang mengubah data (*add*, *edit*, *delete*, *save*) ditolak dalam mode *read-only*.

```
vault> add
[!] Mode backup: tidak bisa tambah data.
vault> edit
[!] Mode backup: tidak bisa edit.
vault> delete
[!] Mode backup: tidak bisa hapus.
vault> save
[!] Mode backup tidak menyimpan perubahan.
vault> █
```

Gambar 4.33 Perubahan Data Ditolak pada Mode Backup

### IV.9 Uji Kegagalan Pemulihan

#### IV.9.1 Recovery Share Salah

Sistem akan menampilkan *error* [!] Format recovery share tidak valid. Harusnya JSON dengan x dan y[!]. jika format *share* salah.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup - tanpa server)
4. Keluar
Pilih: 3
Master password:
Recovery key: {"x":3,"y":"aeEUtWgXcr4Sg2tcCg1ysg=}
[!] Format recovery share tidak valid. Harusnya JSON dengan x dan y.
```

Gambar 4.34 Pengujian Recovery Share Format Salah

Dan sistem akan menampilkan *error* `[!] Dekripsi vault gagal`. Master key tidak valid atau vault rusak. karena *share* tidak valid atau rekonstruksi *master key* gagal.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 3
Master password:
Recovery key: {"x":2,"y":"aeEUtWgXcr4Sg2tcCg1ysg=="}
[!] Dekripsi vault gagal. Master key tidak valid atau vault rusak.
```

Gambar 4.35 Pengujian *Recovery Share* Tidak Valid atau Rusak

#### IV.9.2 Backup Vault Dimodifikasi

*Ciphertext* atau *tag* dimodifikasi pada file `backup_vault.json`.

```
{ } backup_vault.json > { } enc_vault_payload > abc ciphertext
{"enc_vault_payload": {"nonce": "2mVpFWq5Hdo1QycB", "ciphertext": "test", "tag": "+U4I2EEE8bEsHXWYevK4zQ=="}}
```

Gambar 4.36 *Ciphertext* Dimodifikasi

Dekripsi gagal karena GCM *tag authentication* tidak cocok. AES-GCM akan mendeteksi bahwa *ciphertext* sudah dimodifikasi dan *tag* tidak valid. Muncul *error* `[!] Dekripsi vault gagal`. Master key tidak valid atau vault rusak.

```
=== PASSWORD MANAGER (2-of-3 Shamir) ===
1. Setup Vault Baru
2. Buka Vault (Mode Normal)
3. Buka Vault (Mode Backup – tanpa server)
4. Keluar
Pilih: 3
Master password:
Recovery key: {"x":3,"y":"aeEUtWgXcr4Sg2tcCg1ysg=="}
[!] Dekripsi vault gagal. Master key tidak valid atau vault rusak.
```

Gambar 4.37 Pengujian *Backup Vault* Dimodifikasi

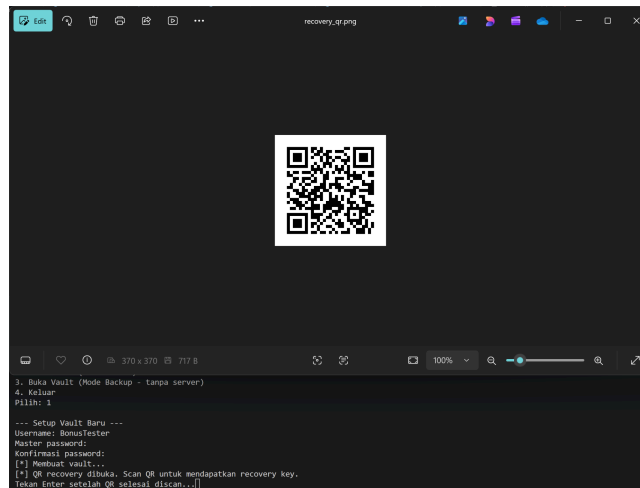
#### IV.9.3 Data Password Tidak Ditampilkan

Ketika dekripsi gagal, sistem tidak akan membuat *VaultSession object*. *User* tidak akan bisa mengakses atau melihat data *password* apapun. Pada kedua kasus di atas, tidak ada data *password* yang ditampilkan.

### IV.10 Uji Kriptografi Visual (bonus)

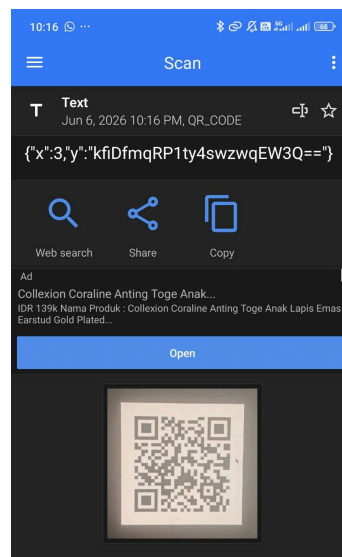
#### IV.10.1 Pembuatan QR Code dari *Recovery Share*

Pembuatan QR Code dilakukan dari *recovery share* yang dihasilkan ketika pengguna membuat *vault* baru.



**Gambar 4.38** *QR Code Recovery Share*

QR Code yang telah dibuat kemudian dipindai menggunakan aplikasi QR scanner. Hasil pemindaian menunjukkan bahwa QR Code berhasil dibaca dan menghasilkan recovery share dalam format JSON.



**Gambar 4.39** Hasil *Scan QR Code Recovery Share*

Selain melalui *QR Code*, program juga menyediakan opsi *fallback* untuk menampilkan *recovery share* dalam bentuk teks melalui CLI. Opsi ini berguna apabila pengguna tidak dapat memindai *QR Code* atau ingin menyimpan *recovery share* secara manual.

```

Tekan Enter setelah QR selesai discan...
Tampilkan recovery key teks sebagai fallback? (y/N): y
=====
RECOVERY KEY - SIMPAN SEKARANG, HANYA TAMPIL SEKALI
=====
{"x":3,"y":"kfiDfmqRP1ty4swzwqEW3Q=="}
=====

```

**Gambar 4.40** *Recovery Key* di CLI sebagai *Fallback* Opsional

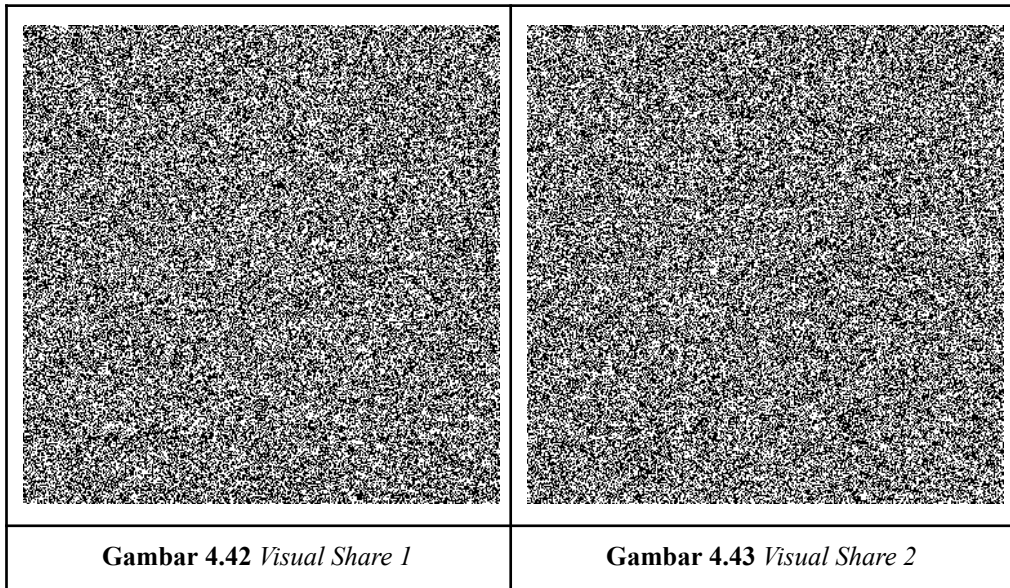
#### IV.10.2 Dua Gambar *Share* Hasil Pembagian QR Code

Setelah QR Code *recovery share* berhasil dibuat, program akan membagi QR Code tersebut menjadi dua visual share. Proses pembagian ini dilakukan menggunakan kriptografi visual, sehingga QR Code asli tidak disimpan secara langsung dalam satu gambar utuh. Program kemudian menghasilkan dua file visual share, yaitu `recovery_visual_share_1.png` dan `recovery_visual_share_2.png`.

```
[+] Visual recovery share dibuat:
1. C:\Users\Carlen\Documents\GitHub\II4021-Kriptografi-Tugas-4\data\recovery_visual_share_1.png
2. C:\Users\Carlen\Documents\GitHub\II4021-Kriptografi-Tugas-4\data\recovery_visual_share_2.png
Simpan kedua file ini terpisah.
[+] Vault berhasil dibuat!
```

**Gambar 4.41** Pembuatan Dua *Visual Share* oleh Program

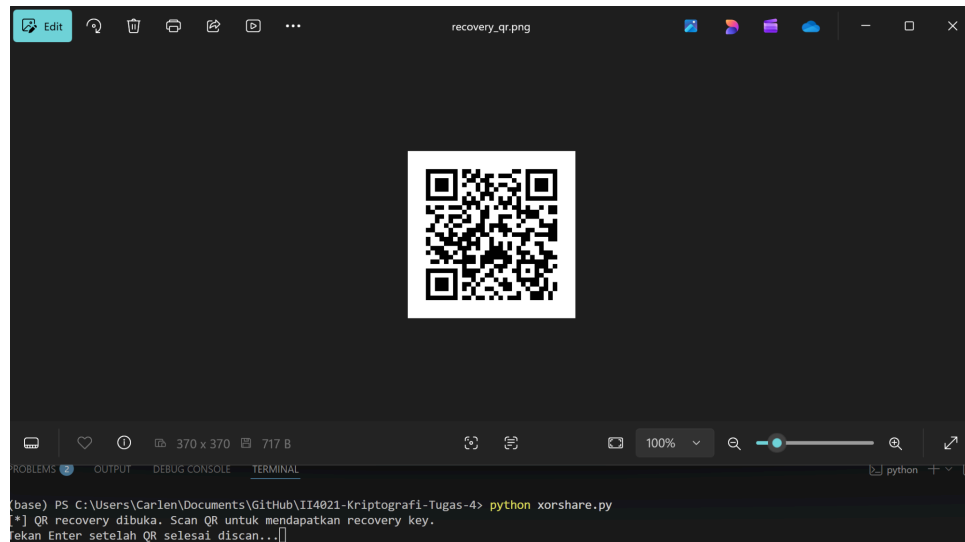
Kedua *visual share* yang dihasilkan tampak seperti gambar noise acak. Masing-masing *share* tidak dapat dipindai sebagai QR Code dan tidak menampilkan informasi *recovery share* secara langsung. Hal ini menunjukkan bahwa satu *visual share* saja tidak cukup untuk memperoleh *recovery share*.



#### IV.10.3 Hasil Penggabungan Kedua Gambar *Share*

Setelah dua *visual share* berhasil dibuat, pengujian dilanjutkan dengan menggabungkan kedua gambar share tersebut. Proses penggabungan dilakukan menggunakan operasi XOR antara `recovery_visual_share_1.png` dan `recovery_visual_share_2.png`. Implementasi penggabungan ini terdapat pada file `xorshare.py`, dan dijalankan melalui perintah

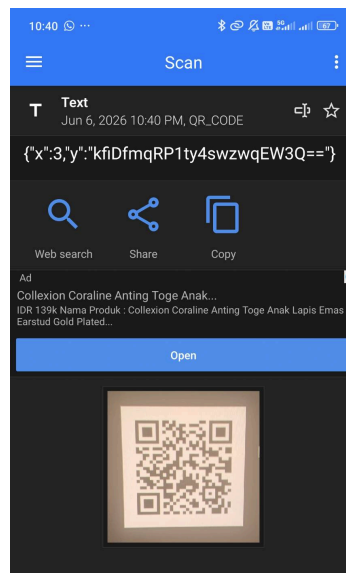
```
python xorshare.py
```



Gambar 4.44 Hasil Penggabungan Kedua *Visual Share*

#### IV.10.4 Validasi *Recovery Code* Hasil Penggabungan Kedua Gambar *Share*

Untuk memastikan bahwa hasil penggabungan dua visual share menghasilkan QR Code yang valid, QR Code hasil rekonstruksi kemudian dipindai menggunakan aplikasi QR scanner. Pengujian ini dilakukan untuk membuktikan bahwa QR Code hasil penggabungan masih dapat dibaca dan menghasilkan recovery share yang sama dengan recovery share awal.



**Gambar 4.45** Hasil *Scan QR Code* dari Penggabungan Dua *Visual Share*

## V. Kesimpulan

Pada tugas ini telah berhasil dibangun sebuah *password manager* terdistribusi berbasis CLI yang memadukan enkripsi AES-128-GCM untuk melindungi isi *vault* dengan skema Shamir Secret Sharing (2,3) untuk mendistribusikan kunci enkripsi. *Master key* sepanjang 16 *byte* tidak pernah disimpan secara utuh, melainkan dipecah menjadi tiga *share* yang ditempatkan di tiga lokasi berbeda, yaitu *local share* di perangkat pengguna yang dienkripsi dengan kunci turunan PBKDF2 dari *master password*, *server share* di basis data SQLite, dan *recovery share* yang dipegang pengguna. Dengan rancangan ini *vault* hanya dapat dibuka apabila minimal dua *share* yang valid berhasil digabungkan.

Hasil pengujian menunjukkan seluruh kasus uji wajib terpenuhi. Pembuatan *vault* membangkitkan *master key* acak dan membaginya dengan benar ke dalam skema (2,3), dan rekonstruksi dari sembarang dua *share* menghasilkan *master key* yang identik. Pada mode normal, kombinasi *local share* dan *server share* berhasil merekonstruksi *master key* serta mendekripsi *vault*, sedangkan mode *backup* tetap dapat membuka *vault* secara *read-only* menggunakan *local share* dan *recovery share* dari salinan lokal terenkripsi tanpa menghubungi server. Perlindungan *local share* terbukti efektif, yaitu *master password* yang salah langsung ditolak pada tahap dekripsi *local share* sebelum sistem menghubungi server.

Prinsip *zero-knowledge* pada server juga terjaga. Server hanya menyimpan *vault* terenkripsi sebagai BLOB beserta *server share*, *nonce*, *authentication tag*, dan metadata pengguna, tanpa kolom *plaintext* apa pun, dan secara aktif menolak permintaan yang menyertakan data sensitif seperti *master key*. Integritas data dijamin oleh *authentication tag* AES-128-GCM, sebagaimana ditunjukkan pada pengujian kegagalan pemulihan: *recovery share* yang salah maupun *backup vault* yang dimodifikasi menyebabkan dekripsi gagal dan tidak ada data *password* yang ditampilkan. Sinkronisasi antara server dan *backup* lokal pun terbukti konsisten melalui kesamaan *nonce vault* pada keduanya setelah setiap penyimpanan.

Fitur bonus kriptografi visual untuk *recovery share* berhasil diimplementasikan dengan skema (2,2). *QR code recovery* dipecah menjadi dua gambar yang masing-masing tampak seperti *noise* acak dan tidak mengungkapkan informasi apa pun, sementara hasil penggabungan keduanya melalui operasi XOR menghasilkan *QR code* yang kembali dapat dipindai dan memuat *recovery share* yang sama.

Secara keseluruhan, tujuan utama sistem tercapai, yaitu memastikan tidak ada satu entitas maupun satu titik penyimpanan yang memiliki cukup informasi untuk membuka *vault* secara mandiri, sehingga kebocoran server atau kehilangan satu perangkat tidak serta-merta mengorbankan keamanan data pengguna. Sebagai pengembangan lanjutan, komunikasi klien–server yang saat ini berjalan pada lingkungan lokal dapat diamankan dengan TLS/HTTPS untuk penerapan nyata, dan mekanisme autentikasi pengguna pada server dapat diperkuat lebih lanjut.

## VI. Daftar Pustaka

- [1] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, hlm. 612–613, 1979.
- [2] R. Munir, "Skema Pembagian Data Rahasia (Secret Sharing Schemes)," Bahan Kuliah II4021 Kriptografi, Program Studi Sistem dan Teknologi Informasi, STEI, Institut Teknologi Bandung, 2025. [Daring]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2025-2026/37-Skema-Pembagian-Data-Rahasia-2025.pdf>
- [3] M. Naor dan A. Shamir, "Visual cryptography," dalam *Advances in Cryptology — EUROCRYPT'94*, LNCS vol. 950, Springer, 1995, hlm. 1–12.
- [4] R. Munir, "Kriptografi Visual (Visual Cryptography), Bagian 1 dan 2," Bahan Kuliah II4021 Kriptografi, Program Studi Sistem dan Teknologi Informasi, STEI, Institut Teknologi Bandung, 2025. [Daring]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2025-2026/38-Kriptografi-Visual-Bagian1-2025.pdf>
- [5] National Institute of Standards and Technology, *FIPS PUB 197: Advanced Encryption Standard (AES)*, Gaithersburg, MD, 2001.
- [6] R. Munir, "Beberapa Block Cipher (Bagian 1): DES dan AES," Bahan Kuliah II4021 Kriptografi, Program Studi Sistem dan Teknologi Informasi, STEI, Institut Teknologi Bandung, 2025. [Daring]. Tersedia: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2025-2026/16-Beberapa-block-cipher-bagian1-2025.pdf>
- [7] M. Dworkin, *NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, National Institute of Standards and Technology, 2007.
- [8] K. Moriarty, B. Kaliski, dan A. Rusch, *RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1*, Internet Engineering Task Force (IETF), 2017.
- [9] Python Software Foundation, "secrets — Generate secure random numbers for managing secrets," *Python 3 Documentation*. [Daring]. Tersedia: <https://docs.python.org/3/library/secrets.html>

## VII. Lampiran

Lampiran 1. Pranala Repositori GitHub

<https://github.com/renathalicia/II4021-Kriptografi-Tugas-4.git>

Lampiran 2. Pranala Video Demo

<https://youtu.be/b1mIDoFcZGk>

Lampiran 3. Pembagian Tugas Anggota Kelompok

**Tabel 7.1** Pembagian Tugas Anggota Kelompok

| Nama                                        | NIM      | Tanggung Jawab                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Samuel Chris Michael<br>Bagasta Simanjuntak | 18223011 | <ul style="list-style-type: none"><li>• Implementasi CLI, alur pengguna, CRUD Password pengguna, mode normal dan <i>backup</i>, implementasi <i>password generator</i>, integrasi keseluruhan</li><li>• Laporan bagian II.1-II.7, III.1-III.2, III.4-III.9, IV.8</li><li>• Video Bagian Pembukaan, Teori dan Penutupan</li></ul> |
| Carlen Asadel Axelle                        | 18223017 | <ul style="list-style-type: none"><li>• Implementasi kriptografi inti, Shamir Secret Sharing, AES-GCM, KDF, CSPRNG, dan bonus <i>visual cryptography</i></li><li>• Struktur Laporan dan Bagian IV.10</li><li>• Video Bagian Demo Program dan Penyuntingan</li></ul>                                                              |
| Audy Alicia Renatha<br>Tirayoh              | 18223097 | <ul style="list-style-type: none"><li>• Implementasi server, API, SQLite, penyimpanan vault terenkripsi, server share, nonce, dan metadata</li><li>• Laporan bagian III.3 dan IV.1 - IV.7, IV.9, dan Kesimpulan</li><li>• Video Bagian Penjelasan Kode</li></ul>                                                                 |